Student Course Management System

COS 201 - Computer Programming I | Lab Assessment

Name: Muhammad SaniAbdulkadir

Matriculation number: 2025/B/CYB/0948

Student ID: 301826989

Email: m.abdulkadir6989@miva.edu.ng

Department: Cyber Security

What the system does

This Java console program keeps a list of courses for a semester. Each course has a code, a title and credit units. The menu lets the user add a course, view the list, search by code, calculate total units, save the list, load a saved list and exit.

How the main features work

The Course class stores and validates one course. CourseManager keeps the courses in an ArrayList and provides the search, total, save and load methods. StudentCourseApp reads input with Scanner and uses a loop and switch statement to handle the seven menu choices.

Course codes are trimmed, converted to uppercase and checked for valid characters. Titles are trimmed and cannot be blank. Units must be whole numbers from 1 to 10; this is a program validation rule. Duplicate course codes are rejected. Searching ignores the letter case used by the user.

Saving writes a UTF-8 text file with the header COURSES_V1 and one tab-separated record per line. The default filename is courses.tsv. Loading replaces the current list only after every record is valid. The menu asks before overwriting an existing file or replacing unsaved changes.

Recursion

The totalUnits method calls sumUnits(start, end). An empty range returns zero, and a range with one course returns that course's units. Larger ranges are split in half, and the two recursive results are added. This reduces the depth of the call stack. Loops are used for the menu, displaying courses, searching and processing file records.

Error handling and validation

The main error cases are non-numeric units, blank fields, duplicate codes, missing files and damaged file records. NumberFormatException handles invalid numeric input. IllegalArgumentException reports invalid course details. IOException reports file failures. A failed load leaves the current courses intact. If input closes, the program exits and warns when changes have not been saved.

Testing results

The automated checks passed: 33 core checks and 30 console checks. They cover course validation, normalized search, duplicate detection, recursive totals, saving and loading, empty lists, UTF-8 text, malformed files, missing paths, menu errors, cancellation and closed input. A 10,000-course fixture also produced the expected total without a stack overflow.

A separate run used the eight courses shown in the supplied course list. It displayed all eight, found COS 201 using lowercase input, calculated 18 units, saved the file and loaded eight courses back. The image below is a browser screenshot of the actual captured console output; the full output is included in program-transcript.txt. These checks demonstrate the tested cases, rather than guaranteeing every possible input.



Run: bash scripts/build.sh, then java -cp build StudentCourseApp. Tests: bash scripts/test.sh. Requires Java 21; the cloud environment also has the Eclipse compiler available.